TEMA: L2. Arhitectura sistemului federativ de integrare [FDB Oracle]

Sisteme Informaționale Integrate
Sistem federativ de integrare a datelor olimpice

Indicativ echipă: **SIA04**

IMPLEMENTARE ACCESS SURSE DE DATE EXTERNE

Baza de date centrală: Oracle Cloud Autonomous Database 26ai (Always Free Tier)
Schema de integrare/mediere: FDBO
Schema sursă referință: OLY_REF
Total view-uri de acces definite: 9

DS	Mecanism de acces	View-uri	Rânduri
DS_1 Oracle	GRANT SELECT cross-schema	4	1.113
DS_2 PostgreSQL	ETL Python (echivalent DB Link)	2	406.687
DS_3 JSON/MongoDB	JSON_TABLE (Oracle SQL/JSON)	2	28.302
DS_4 CSV	Tabel local (echivalent Oracle Loader)	1	50.000
TOTAL		**9**	**486.102**

Cuprins

1 DS_1: Oracle – Date de referință olimpice 3
1.1 Nume și descriere sursă de date . . . 3
1.2 Descriere mecanism de acces . . . 3
1.3 Implementare – Creare utilizatori și drepturi . . . 3
1.4 Implementare – Acordare drepturi SELECT . . . 4
1.5 Definiție structuri de acces – View-uri FDBO . . . 4
1.5.1 FDBO_SPORTS_V . . . 4
1.5.2 FDBO_EVENTS_V . . . 4
1.5.3 FDBO_GAMES_V . . . 4
1.5.4 FDBO_COUNTRIES_V . . . 4
1.6 Script complet . . . 4

2 DS_2: PostgreSQL – Date tranzacționale atleți și rezultate 5
2.1 Nume și descriere sursă de date . . . 5
2.2 Descriere mecanism de acces . . . 5
2.3 Implementare – DB Link heterogen (Oracle local) . . . 5
2.3.1 Configurare gateway DG4ODBC . . . 5
2.3.2 Configurare listener.ora . . . 6
2.3.3 Configurare tnsnames.ora . . . 6
2.3.4 Creare DB Link . . . 6
2.4 Implementare – Tabele locale Oracle (destinație ETL) . . . 6
2.5 Definiție structuri de acces – View-uri FDBO . . . 7
2.5.1 FDBO_PG_ATHLETES_V . . . 7
2.5.2 FDBO_PG_RESULTS_V . . . 7
2.5.3 View-uri DB Link remote (alternativă cu Oracle local) . . . 7
2.6 Scripturi complete . . . 8

3 DS_3: MongoDB/JSON – Documente NoSQL cu agregări medalii 9
3.1 Nume și descriere sursă de date . . . 9
3.2 Descriere mecanism de acces . . . 9
3.3 Implementare – Creare tabel de stocare JSON . . . 9
3.4 Definiție structuri de acces – View-uri JSON_TABLE . . . 10
3.4.1 FDBO_JSON_ATHLETE_MEDALS_V (28.251 rânduri) . . . 10
3.4.2 FDBO_JSON_GAME_SUMMARY_V (51 rânduri) . . . 10
3.5 Scripturi complete . . . 11

4 DS_4: CSV – Fișier plat cu date denormalizate 12
4.1 Nume și descriere sursă de date . . . 12
4.2 Descriere mecanism de acces . . . 12
4.3 Implementare – Creare tabel destinație . . . 12
4.4 Definiție structură de acces – View FDBO . . . 13
4.5 Scripturi complete . . . 13

5 Sumar – Toate view-urile de acces din schema FDBO 14
5.1 Verificare view-uri . . . 14
5.2 Arhitectura de acces – Diagrama logică . . . 14

1 DS_1: Oracle – Date de referință olimpice

1.1 Nume și descriere sursă de date

Parametru	Valoare
Nume sursă	Baza de date Oracle – schema OLY_REF
Format/Tip de acces	SQL
Tabele sursă	SPORTS (66), EVENTS (765), GAMES (52), COUNTRIES (230)
View-uri de acces	FDBO_SPORTS_V, FDBO_EVENTS_V, FDBO_GAMES_V, FDBO_COUNTRIES_V

1.2 Descriere mecanism de acces

Mecanism: *GRANT SELECT cross-schema* (acces direct la tabelele din schema OLY_REF, fără DB Link)

Sursa de date DS_1 se află pe **aceeași instanță Oracle Cloud** cu schema de integrare FDBO. Accesul se realizează prin:

1. **Creare utilizator OLY_REF** cu drepturi de CREATE TABLE, CREATE VIEW
2. **Creare tabele** SPORTS, EVENTS, GAMES, COUNTRIES în schema OLY_REF
3. **Acordare drepturi** GRANT SELECT pe fiecare tabel către utilizatorul FDBO
4. **Definire VIEW-uri** în schema FDBO care referențiază direct `OLY_REF.tabel`

Aceasta este modalitatea standard Oracle de **acces cross-schema** – echivalentul unui DB Link local, dar fără overhead-ul de rețea deoarece ambele scheme rezidă pe aceeași instanță.

1.3 Implementare – Creare utilizatori și drepturi

Listing 1: CLOUD_01_CREATE_USERS.sql – Creare utilizatori

```sql
CREATE USER OLY_REF IDENTIFIED BY "Oracle_1234#"
DEFAULT TABLESPACE DATA
TEMPORARY TABLESPACE TEMP
QUOTA UNLIMITED ON DATA;

GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO OLY_REF;

CREATE USER FDBO IDENTIFIED BY "Oracle_1234#"
DEFAULT TABLESPACE DATA
TEMPORARY TABLESPACE TEMP
QUOTA UNLIMITED ON DATA;

GRANT CONNECT, RESOURCE TO FDBO;
GRANT CREATE VIEW TO FDBO;
GRANT CREATE DATABASE LINK TO FDBO;
```

1.4 Implementare – Acordare drepturi SELECT

Listing 2: GRANT SELECT – acces cross-schema OLY_REF → FDBO

```
GRANT SELECT ON OLY_REF.SPORTS    TO FDBO;
GRANT SELECT ON OLY_REF.EVENTS    TO FDBO;
GRANT SELECT ON OLY_REF.GAMES     TO FDBO;
GRANT SELECT ON OLY_REF.COUNTRIES TO FDBO;
```

1.5 Definiție structuri de acces – View-uri FDBO

1.5.1 FDBO_SPORTS_V

Listing 3: View acces sporturi

```
CREATE OR REPLACE VIEW FDBO.FDBO_SPORTS_V AS
SELECT sport_id, sport_name, 'Oracle/OLY_REF' AS data_source
FROM OLY_REF.SPORTS;
```

1.5.2 FDBO_EVENTS_V

Listing 4: View acces evenimente (cu JOIN la SPORTS)

```
CREATE OR REPLACE VIEW FDBO.FDBO_EVENTS_V AS
SELECT e.event_id, e.sport_id, e.event_name,
       s.sport_name,
       'Oracle/OLY_REF' AS data_source
FROM OLY_REF.EVENTS e
JOIN OLY_REF.SPORTS s ON e.sport_id = s.sport_id;
```

1.5.3 FDBO_GAMES_V

Listing 5: View acces ediții olimpice

```
CREATE OR REPLACE VIEW FDBO.FDBO_GAMES_V AS
SELECT game_id, games_name, year, season, city,
       'Oracle/OLY_REF' AS data_source
FROM OLY_REF.GAMES;
```

1.5.4 FDBO_COUNTRIES_V

Listing 6: View acces țări/comitete olimpice

```
CREATE OR REPLACE VIEW FDBO.FDBO_COUNTRIES_V AS
SELECT noc, region, notes,
       'Oracle/OLY_REF' AS data_source
FROM OLY_REF.COUNTRIES;
```

1.6 Script complet

Fișier: CLOUD_06_FDBO_ACCESS_VIEWS.sql (secțiunea DS_1)

2 DS_2: PostgreSQL – Date tranzacționale atleți și rezultate

2.1 Nume și descriere sursă de date

Parametru	Valoare
Nume sursă	PostgreSQL 16 local – baza `olympics_db`
Format/Tip de acces	SQL
Tabele sursă (PG)	athletes (135.571), results (271.116)
Tabele destinație (Oracle)	FDBO.PG_ATHLETES, FDBO.PG_RESULTS
View-uri de acces	FDBO_PG_ATHLETES_V, FDBO_PG_RESULTS_V

2.2 Descriere mecanism de acces

Mecanism principal: *ETL Python (echivalent DB Link heterogen)*
Mecanism teoretic: *DATABASE LINK heterogen Oracle → PostgreSQL via DG4ODBC*

Accesul la sursa PostgreSQL se realizează printr-un **proces ETL** implementat în Python (`load_to_oracle.py`) care:

1. Se conectează la PostgreSQL local (`localhost:5432/olympics_db`)
2. Extrage datele din tabelele `athletes` și `results`
3. Transformă schema (redenumire coloane, mapare tipuri)
4. Încarcă datele în tabelele Oracle Cloud `FDBO.PG_ATHLETES` și `FDBO.PG_RESULTS`

De ce nu DB Link direct? Oracle Cloud Autonomous Database **nu suportă** Heterogeneous Services (DG4ODBC) necesare pentru DB Link către baze non-Oracle. ETL-ul Python realizează **funcțional același lucru** – aduce datele din PostgreSQL în Oracle pentru a fi accesate prin view-uri FDBO.

Script-ul DB Link heterogen a fost totuși implementat pentru demonstrare teoretică (08.FDBO_DBLINK_PG.sql, 08b.FDBO_DBLINK_PG_FULL_CONFIG.sql).

2.3 Implementare – DB Link heterogen (Oracle local)

2.3.1 Configurare gateway DG4ODBC

Listing 7: initpg_hs.ora – Configurare Heterogeneous Services

```
-- Fisier: $ORACLE_HOME/hs/admin/initpg_hs.ora
-- HS_FDS_CONNECT_INFO = pg_olympics   (ODBC DSN)
-- HS_FDS_TRACE_LEVEL = OFF
-- HS_FDS_SHAREABLE_NAME = C:\psqlodbc\psqlodbc35w.dll
```

2.3.2 Configurare listener.ora

Listing 8: listener.ora – Intrare SID pentru gateway PostgreSQL

```
-- SID_LIST_LISTENER =
--   (SID_LIST =
--     (SID_DESC =
--       (SID_NAME = pg_hs)
--       (ORACLE_HOME = C:\app\oracle\product\21c)
--       (PROGRAM = dg4odbc)
--     )
--   )
```

2.3.3 Configurare tnsnames.ora

Listing 9: tnsnames.ora – Intrare TNS pentru gateway

```
-- pg_hs =
--   (DESCRIPTION =
--     (ADDRESS = (PROTOCOL=TCP)(HOST=localhost)(PORT=1522))
--     (CONNECT_DATA = (SID = pg_hs))
--     (HS = OK)
--   )
```

2.3.4 Creare DB Link

Listing 10: 08.FDBO_DBLINK_PG.sql – Creare DB Link heterogen

```sql
CREATE DATABASE LINK pg_olympics_link
CONNECT TO "olympics" IDENTIFIED BY "olympics"
USING 'pg_hs';

SELECT COUNT(*) FROM "athletes"@pg_olympics_link;
SELECT COUNT(*) FROM "results"@pg_olympics_link;
```

2.4 Implementare – Tabele locale Oracle (destinație ETL)

Listing 11: CLOUD_04_FDBO_TABLES.sql – Tabele pentru date PostgreSQL

```sql
CREATE TABLE FDBO.PG_ATHLETES (
    athlete_id    NUMBER(10)     PRIMARY KEY,
    full_name     VARCHAR2(300) NOT NULL,
    sex           VARCHAR2(1),
    birth_year    NUMBER(4),
    birth_place   VARCHAR2(200),
    height_cm     NUMBER(5,1),
    weight_kg     NUMBER(5,1),
    noc           VARCHAR2(10),
    data_source   VARCHAR2(20) DEFAULT 'PostgreSQL'
);
```

```

CREATE TABLE FDBO.PG_RESULTS (
    result_id     NUMBER(10)     PRIMARY KEY,
    athlete_id    NUMBER(10),
    game_id       NUMBER(10),
    event_id      NUMBER(10),
    medal         VARCHAR2(10),
    age           NUMBER(3),
    team          VARCHAR2(200),
    noc           VARCHAR2(10),
    data_source   VARCHAR2(20) DEFAULT 'PostgreSQL',
    CONSTRAINT FK_PG_RES_ATHLETE
        FOREIGN KEY (athlete_id)
        REFERENCES FDBO.PG_ATHLETES(athlete_id)
);

CREATE INDEX FDBO.IDX_PG_RES_ATHLETE ON FDBO.PG_RESULTS(athlete_id)
    ;
CREATE INDEX FDBO.IDX_PG_RES_GAME    ON FDBO.PG_RESULTS(game_id);
CREATE INDEX FDBO.IDX_PG_RES_EVENT   ON FDBO.PG_RESULTS(event_id);
CREATE INDEX FDBO.IDX_PG_RES_MEDAL   ON FDBO.PG_RESULTS(medal);
CREATE INDEX FDBO.IDX_PG_ATH_NOC     ON FDBO.PG_ATHLETES(noc);
```

2.5 Definiție structuri de acces – View-uri FDBO

2.5.1 FDBO_PG_ATHLETES_V

Listing 12: View acces atleți (proveniți din PostgreSQL)

```
CREATE OR REPLACE VIEW FDBO.FDBO_PG_ATHLETES_V AS
SELECT athlete_id, full_name, sex, birth_year, birth_place,
       height_cm, weight_kg, noc,
       'PostgreSQL' AS data_source
FROM FDBO.PG_ATHLETES;
```

2.5.2 FDBO_PG_RESULTS_V

Listing 13: View acces rezultate (provenite din PostgreSQL)

```
CREATE OR REPLACE VIEW FDBO.FDBO_PG_RESULTS_V AS
SELECT result_id, athlete_id, game_id, event_id,
       medal, age, team, noc,
       'PostgreSQL' AS data_source
FROM FDBO.PG_RESULTS;
```

2.5.3 View-uri DB Link remote (alternativă cu Oracle local)

Listing 14: View-uri remote via DB Link (necesită Oracle local)

```
CREATE OR REPLACE VIEW FDBO.PG_ATHLETES_V AS
```

```sql
SELECT
    "athlete_id" AS athlete_id,
    "name"       AS full_name,
    "sex"        AS sex,
    "height"     AS height_cm,
    "weight"     AS weight_kg,
    'PostgreSQL/DBLink' AS data_source
FROM "athletes"@pg_olympics_link;

CREATE OR REPLACE VIEW FDBO.PG_RESULTS_V AS
SELECT
    "result_id"  AS result_id,
    "athlete_id" AS athlete_id,
    "game_id"    AS game_id,
    "event_id"   AS event_id,
    "medal"      AS medal,
    "age"        AS age,
    "team"       AS team,
    "noc"        AS noc,
    'PostgreSQL/DBLink' AS data_source
FROM "results"@pg_olympics_link;
```

2.6 Scripturi complete

Fișiere: 08.FDBO_DBLINK_PG.sql, 08b.FDBO_DBLINK_PG_FULL_CONFIG.sql, CLOUD_04_FDBO_TABLES.sql, CLOUD_06_FDBO_ACCESS_VIEWS.sql, load_to_oracle.py

3 DS_3: MongoDB/JSON – Documente NoSQL cu agregări medalii

3.1 Nume și descriere sursă de date

Parametru	Valoare
Nume sursă	Colecție MongoDB (simulare) – documente JSON
Format/Tip de acces	JSON
Fișiere sursă	athlete_medals_docs.jsonl (28.251), game_summary_docs.jsonl (51)
Tabel destinație	FDBO.JSON_MEDAL_DOCS (28.302 rânduri)
View-uri de acces	FDBO_JSON_ATHLETE_MEDALS_V, FDBO_JSON_GAME_SUMMARY_V

3.2 Descriere mecanism de acces

Mecanism: *JSON_TABLE (Oracle SQL/JSON)* – funcția nativă Oracle pentru extragerea datelor structurate din documente JSON

Accesul la datele JSON se realizează prin:

1. **Stocare documente** în tabelul `FDBO.JSON_MEDAL_DOCS` cu coloana `json_data` de tip CLOB cu constraint `IS JSON`
2. **Discriminare tip** prin coloana `doc_type`: `'athlete_medal'` sau `'game_summary'`
3. **Extragere structurată** prin `JSON_TABLE()` care transformă documentele JSON în coloane relaționale (virtual relational view)
4. **Mapare câmpuri** JSON → coloane SQL prin clauza `PATH '$.camp'`

Aceasta este abordarea Oracle nativă echivalentă cu:

- MongoDB Connector for Oracle (funcționalitate similară)
- Oracle SODA (Simple Oracle Document Access) pentru operații NoSQL
- Oracle Database API for MongoDB (compatibilitate MongoDB pe Oracle 23ai)

3.3 Implementare – Creare tabel de stocare JSON

Listing 15: CLOUD_04_FDBO_TABLES.sql – Tabel JSON cu constraint IS JSON

```sql
CREATE TABLE FDBO.JSON_MEDAL_DOCS (
    doc_id        NUMBER(10) GENERATED ALWAYS AS IDENTITY
                  PRIMARY KEY,
    doc_type      VARCHAR2(30)  NOT NULL,
    json_data     CLOB CHECK (json_data IS JSON),
    created_at    TIMESTAMP DEFAULT SYSTIMESTAMP,
    data_source   VARCHAR2(20) DEFAULT 'MongoDB/JSON'
```

```
);

CREATE INDEX FDBO.IDX_JSON_DOC_TYPE
    ON FDBO.JSON_MEDAL_DOCS(doc_type);
```

3.4 Definiție structuri de acces – View-uri JSON_TABLE

3.4.1 FDBO_JSON_ATHLETE_MEDALS_V (28.251 rânduri)

Listing 16: View acces medalii atleți – JSON_TABLE cu 7 coloane extrase

```
CREATE OR REPLACE VIEW FDBO.FDBO_JSON_ATHLETE_MEDALS_V AS
SELECT
    j.doc_id,
    jt.athlete_name,
    jt.total_medals,
    jt.gold_medals,
    jt.silver_medals,
    jt.bronze_medals,
    jt.sports_count,
    jt.games_count,
    'MongoDB/JSON' AS data_source
FROM FDBO.JSON_MEDAL_DOCS j,
     JSON_TABLE(j.json_data, '$'
         COLUMNS (
             athlete_name  VARCHAR2(300) PATH '$.name',
             total_medals  NUMBER(5)     PATH '$.total_medals',
             gold_medals   NUMBER(5)     PATH '$.gold',
             silver_medals NUMBER(5)     PATH '$.silver',
             bronze_medals NUMBER(5)     PATH '$.bronze',
             sports_count  NUMBER(5)     PATH '$.sports_count',
             games_count   NUMBER(5)     PATH '$.games_count'
         )
     ) jt
WHERE j.doc_type = 'athlete_medal';
```

3.4.2 FDBO_JSON_GAME_SUMMARY_V (51 rânduri)

Listing 17: View acces statistici ediții – JSON_TABLE cu 8 coloane extrase

```
CREATE OR REPLACE VIEW FDBO.FDBO_JSON_GAME_SUMMARY_V AS
SELECT
    j.doc_id,
    jt.games_name,
    jt.year,
    jt.season,
    jt.city,
    jt.total_athletes,
    jt.total_entries,
    jt.countries_count,
```

```sql
    jt.events_count,
    'MongoDB/JSON' AS data_source
FROM FDBO.JSON_MEDAL_DOCS j,
     JSON_TABLE(j.json_data, '$'
         COLUMNS (
             games_name      VARCHAR2(50)  PATH '$.games_name',
             year            NUMBER(4)     PATH '$.year',
             season          VARCHAR2(10)  PATH '$.season',
             city            VARCHAR2(100) PATH '$.city',
             total_athletes  NUMBER(10)    PATH '$.total_athletes',
             total_entries   NUMBER(10)    PATH '$.total_entries',
             countries_count NUMBER(5)     PATH '$.countries_count',
             events_count    NUMBER(5)     PATH '$.events_count'
         )
     ) jt
WHERE j.doc_type = 'game_summary';
```

3.5 Scripturi complete

Fișiere: CLOUD_04_FDBO_TABLES.sql, CLOUD_06_FDBO_ACCESS_VIEWS.sql, load_to_oracle.py

4 DS_4: CSV – Fișier plat cu date denormalizate

4.1 Nume și descriere sursă de date

Parametru	Valoare
Nume sursă	Fișier CSV denormalizat – `athlete_events_enriched.csv`
Format/Tip de acces	CSV
Fișier sursă	csv_data/athlete_events_enriched.csv (50.000 rânduri)
Tabel destinație	FDBO.CSV_ATHLETE_EVENTS (50.000 rânduri)
View de acces	FDBO_CSV_EVENTS_V

4.2 Descriere mecanism de acces

Mecanism: *Tabel local + ETL Python (echivalent Oracle External Table / SQL*Loader)*

Accesul la fișierul CSV se realizează prin:

1. **Citire CSV** cu biblioteca Python `pandas` din fișierul `athlete_events_enriched.csv`
2. **Transformare** – curățare valori NULL, conversie tipuri, mapare coloane
3. **Încărcare bulk** în tabelul `FDBO.CSV_ATHLETE_EVENTS` prin `oracledb.Cursor.executemany()`
4. **Acces** prin view-ul `FDBO_CSV_EVENTS_V`

Aceasta este echivalentul mecanismelor Oracle native:

- **Oracle External Table** – `CREATE TABLE ... ORGANIZATION EXTERNAL` cu ORACLE_LOADER
- **SQL*Loader** – utilitarul clasic de încărcare date din fișiere plate
- **DBMS_CLOUD.COPY_DATA** – funcția Oracle Cloud pentru import din Object Storage

4.3 Implementare – Creare tabel destinație

Listing 18: CLOUD_04_FDBO_TABLES.sql – Tabel pentru date CSV

```sql
CREATE TABLE FDBO.CSV_ATHLETE_EVENTS (
    row_id          NUMBER(10) GENERATED ALWAYS AS IDENTITY
                    PRIMARY KEY,
    athlete_name    VARCHAR2(300),
    sex             VARCHAR2(1),
    age             NUMBER(3),
    height          NUMBER(5,1),
    weight          NUMBER(5,1),
    team            VARCHAR2(200),
    noc             VARCHAR2(10),
```

```
    games           VARCHAR2(50),
    year            NUMBER(4),
    season          VARCHAR2(10),
    city            VARCHAR2(100),
    sport           VARCHAR2(100),
    event           VARCHAR2(200),
    medal           VARCHAR2(10),
    region          VARCHAR2(200),
    data_source     VARCHAR2(20) DEFAULT 'CSV'
);

CREATE INDEX FDBO.IDX_CSV_NOC   ON FDBO.CSV_ATHLETE_EVENTS(noc);
CREATE INDEX FDBO.IDX_CSV_YEAR  ON FDBO.CSV_ATHLETE_EVENTS(year);
CREATE INDEX FDBO.IDX_CSV_MEDAL ON FDBO.CSV_ATHLETE_EVENTS(medal);
CREATE INDEX FDBO.IDX_CSV_SPORT ON FDBO.CSV_ATHLETE_EVENTS(sport);
```

4.4 Definiție structură de acces – View FDBO

Listing 19: View acces date CSV (16 coloane + marcaj sursă)

```
CREATE OR REPLACE VIEW FDBO.FDBO_CSV_EVENTS_V AS
SELECT row_id, athlete_name, sex, age, height, weight,
       team, noc, games, year, season, city,
       sport, event, medal, region,
       'CSV' AS data_source
FROM FDBO.CSV_ATHLETE_EVENTS;
```

4.5 Scripturi complete

Fișiere: CLOUD_04_FDBO_TABLES.sql, CLOUD_06_FDBO_ACCESS_VIEWS.sql, load_to_oracle.py

5 Sumar – Toate view-urile de acces din schema FDBO

#	View FDBO	Sursă	Mecanism	Rânduri
1	FDBO_SPORTS_V	OLY_REF (Oracle)	GRANT SELECT	66
2	FDBO_EVENTS_V	OLY_REF (Oracle)	GRANT SELECT	765
3	FDBO_GAMES_V	OLY_REF (Oracle)	GRANT SELECT	52
4	FDBO_COUNTRIES_V	OLY_REF (Oracle)	GRANT SELECT	230
5	FDBO_PG_ATHLETES_V	PostgreSQL	ETL Python	135.571
6	FDBO_PG_RESULTS_V	PostgreSQL	ETL Python	271.116
7	FDBO_JSON_ATHLETE_MEDALS_V	MongoDB/JSON	JSON_TABLE	28.251
8	FDBO_JSON_GAME_SUMMARY_V	MongoDB/JSON	JSON_TABLE	51
9	FDBO_CSV_EVENTS_V	CSV	Tabel local	50.000
	TOTAL			**486.102**

5.1 Verificare view-uri

Listing 20: Interogare verificare view-uri FDBO

```
SELECT view_name
FROM all_views
WHERE owner = 'FDBO'
ORDER BY view_name;
```

5.2 Arhitectura de acces – Diagrama logică

```
+------------------+      +------------------+
| DS_1: Oracle     |      | DS_2: PostgreSQL |
| OLY_REF schema   |      | localhost:5432   |
| (SPORTS, EVENTS, |      | (athletes,       |
|  GAMES, COUNTRIES)|     | results)         |
+--------+---------+      +--------+---------+
         |                         |
   GRANT SELECT              ETL Python /
   cross-schema              DB Link (DG4ODBC)
         |                         |
         v                         v
+-------------------------------------------------+
|         FDBO -- Schema de Integrare             |
|                                                  |
|  FDBO_SPORTS_V     FDBO_PG_ATHLETES_V           |
```

```
|  FDBO_EVENTS_V     FDBO_PG_RESULTS_V           |
|  FDBO_GAMES_V      FDBO_JSON_ATHLETE_MEDALS_V  |
|  FDBO_COUNTRIES_V  FDBO_JSON_GAME_SUMMARY_V    |
|                    FDBO_CSV_EVENTS_V            |
|                                                 |
|            (9 VIEW-uri de ACCES)                |
+-------------------------------------------------+
         ^                            ^
   JSON_TABLE                 Tabel local /
   (SQL/JSON)                 Oracle Loader
         |                            |
+--------+---------+      +--------+---------+
| DS_3: JSON/Mongo |      | DS_4: CSV File   |
| JSON_MEDAL_DOCS  |      | athlete_events_  |
| (CLOB IS JSON)   |      | enriched.csv     |
+------------------+      +------------------+
```